QUESTIONS 20 / 20 completed

Employee

EMPLOYEE

Write a Java program to implement a basic employee management system using inheritance. Create a parent class called Employee with attributes such as name, id, and salary. Then, create child classes such as Manager and SalesPerson that

PROGRAM EDITOR

Java

Run Save

```java
import java.util.*;
class Employee {
    protected String name;
    protected int id;
    protected double salary;
    public Employee(String name, int id, double salary) {
        this.name = name;
        this.id = id;
        this.salary = salary;
    }
    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Salary: " + salary);
    }
    public void raiseSalary(double amount) {
        if (amount > 0) {
            salary += amount;
        }
    }
    public double getSalary() {
```

OUTPUT

INPUT

Your INPUT goes here!

QUESTIONS 20 / 20 completed

Student-Const

STUDENT-CONST

Write a Java program to create a class called Student with a constructor that takes in a name, ID number, and a list of grades, and methods to calculate and return the student's average grade and letter grade.

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        Student s = new Student("Kaniga", 192565010, 850, 1000);
        s.display();
    }
}
class Student {
    String name;
    int id;
    int marks;
    int maxMarks;
    Student(String name, int id, int marks, int maxMarks) {
        this.name = name;
        this.id = id;
        this.marks = marks;
        this.maxMarks = maxMarks;
    }
    char calculateGrade() {
        double per = (marks * 100.0) / maxMarks;
        if (per >= 90) return 'A';
        else if (per >= 80) return 'B';
```

OUTPUT

INPUT

Your IN
here!

QUESTIONS 20 / 20 completed

Car-Const

CAR-CONST

Write a Java program to create a class called Car with a constructor that takes in the make, model, and year of the car, and a method to print out the car's make, model, and year.

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        Car c = new Car("Toyota", 2020);
        c.display();
    }
}
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }
    void display() {
        System.out.println("Model: " + model + ", Year: " + year);
    }
}
```

OUTPUT

```
Model: Toyota, Year: 2020
```

INPUT

Your INPUT goes here!

QUESTIONS 20 / 20 completed

Bank-Const

BANK-CONST

Write a Java program to create a class called BankAccount with a constructor that takes in an account number and an initial balance, and methods to deposit and withdraw money from the account.

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("Kaniga Sri", 1000.0);
        account.display();
        account.deposit(500.0);
        account.withdraw(200.0);
        account.withdraw(2000.0);
        account.display();
    }
}
class BankAccount {
    String accountHolder;
    double balance;
    BankAccount(String name, double initialBalance) {
        this.accountHolder = name;
        this.balance = initialBalance;
    }
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: " + amount);
```

OUTPUT

```
Holder: Kaniga Sri | Balance: 1000.0
```

INPUT

Your here!

QUESTIONS 20 / 20 completed

Person

PERSON

Write a Java program to create a class called Person with a constructor that takes in a name and age, and a method to print out the person's name and age.

PROGRAM EDITOR

Java

Run

```java
public class Main {
    public static void main(String[] args) {
        Person p1 = new Person("Kaniga", 20);
        p1.display();
    }
}
class Person {
    String name;
    int age;
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

OUTPUT

QUESTIONS 20 / 20 completed

Rect-Const

RECT-CONST

Write a Java program to create a class called Rectangle with a constructor that takes in the length and width of the rectangle, and a method to calculate and return the area of the rectangle.

PROGRAM EDITOR

Java

```java
public class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(10, 5);
        System.out.println("Area = " + r.area());
    }
}
class Rectangle {
    int length, width;
    Rectangle(int length, int width) {
        this.length = length;
        this.width = width;
    }
    int area() {
        return length * width;
    }
}
```

OUTPUT

```
Area = 50
```

QUESTIONS 20 / 20 completed

Overriding

OVERRIDING

Write a Java program to demonstrate method overriding with a simple calculator. Create a parent class called Calculator with methods such as add(), subtract(), multiply(), and divide(). Then, create a child class called ScientificCalculator

PROGRAM EDITOR

Java

```java
public class Main {
    public static void main(String[] args) {
        Calculator basic = new Calculator();
        Calculator advanced = new AdvancedCalculator();
        System.out.print("Basic Calculator: ");
        basic.calculate(10, 5);
        System.out.print("Advanced Calculator: ");
        advanced.calculate(10, 5);
    }
}
class Calculator {
    void calculate(int a, int b) {
        System.out.println("Result: " + (a + b));
    }
}
class AdvancedCalculator extends Calculator {
    void calculate(int a, int b) {
        int sum = a + b;
        int product = a * b;
        System.out.println("Sum: " + sum + ", Product: " + pro
    }
```

OUTPUT

QUESTIONS 20 / 20 completed

Overloading

OVERLOADING

Write a Java program to demonstrate method overloading with variable arguments. Create a method called sum() that takes in a variable number of integers and returns their sum. Then, create an overloaded version of the sum() method that takes in a variable

PROGRAM EDITOR

```java
public class Main {
    public static void main(String[] args) {
        System.out.println(sum(10, 20));
        System.out.println(sum(1, 2, 3));
        System.out.println(sum(1.5, 2.5));
    }
    static int sum(int a, int b) {
        return a + b;
    }
    static int sum(int a, int b, int c) {
        return a + b + c;
    }
    static double sum(double a, double b) {
        return a + b;
    }
}
```

OUTPUT

```
30
```

QUESTIONS 20 / 20 completed

Animal-2

ANIMAL-2

Write a Java program to demonstrate polymorphism with abstract classes. Create an abstract class called Animal with abstract methods such as eat() and sleep(). Then, create child classes such as Dog and Cat that inherit from the Animal class and

PROGRAM EDITOR

Java

Run

```java
public class Main {
    public static void main(String[] args) {
        Animal[] animals = {
            new Lion(),
            new Elephant(),
            new Snake(),
            new Dog()
        };
        for (Animal a : animals) {
            a.makeSound();
        }
    }
}
class Animal {
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}
class Lion extends Animal {
    void makeSound() {
        System.out.println("Lion: Roar!");
```

OUTPUT

```
Lion: Roar!
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

QUESTIONS 20 / 20 completed

Animal-2

ANIMAL-2

Write a Java program to demonstrate polymorphism with abstract classes. Create an abstract class called Animal with abstract methods such as eat() and sleep(). Then, create child classes such as Dog and Cat that inherit from the Animal class and

PROGRAM EDITOR

Java

Run

```java
public class Main {
    public static void main(String[] args) {
        Animal[] animals = {
            new Lion(),
            new Elephant(),
            new Snake(),
            new Dog()
        };
        for (Animal a : animals) {
            a.makeSound();
        }
    }
}
class Animal {
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}
class Lion extends Animal {
    void makeSound() {
        System.out.println("Lion: Roar!");
```

OUTPUT

```
Lion: Roar!
```

QUESTIONS 20 / 20 completed

Draw

DRAW

Write a Java program to demonstrate polymorphism with interfaces. Create an interface called Drawable with a method called draw(). Then, create classes such as Circle and Square that implement the Drawable interface and have their own unique

PROGRAM EDITOR

Java

```java
public class Main {
    public static void main(String[] args) {
        Animal soundMaker;
        soundMaker = new Dog();
        soundMaker.makeSound();
        soundMaker = new Cat();
        soundMaker.makeSound();
        soundMaker = new Bird();
        soundMaker.makeSound();
    }
}
interface Animal {
    void makeSound();
}
class Dog implements Animal {
    public void makeSound() {
        System.out.println("Dog says: Woof Woof");
    }
}
class Cat implements Animal {
    public void makeSound() {
```

OUTPUT

```
Dog says: Woof Woof
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

QUESTIONS 20 / 20 completed

Shape-2

SHAPE-2

Write a Java program to implement a basic shape hierarchy using polymorphism. Create a parent class called Shape with attributes such as area and perimeter. Then, create child classes such as Circle and Rectangle that inherit from the Shape class and have their own

PROGRAM EDITOR

Java

Run

Save

```java
public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Square(5),
            new Triangle(4),
            new Circle(3)
        };
        for (Shape s : shapes) {
            System.out.println(s.getClass().getSimpleName() + " Area = " + s.area());
        }
    }
}
abstract class Shape {
    abstract double area();
}
class Square extends Shape {
    double side;
    Square(double side) { this.side = side; }
    double area() { return side * side; }
}
class Triangle extends Shape {
```

OUTPUT

```
Square Area = 25.0
```

QUESTIONS 20 / 20 completed

Rental

RENTAL

Write a Java program to implement a basic car rental system using inheritance. Create a parent class called Vehicle with attributes such as make and model. Then, create child classes such as Sedan and SUV that inherit from the Vehicle class and have their own unique

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new Car("Sedan", 80);
        Vehicle v2 = new Bike("Sport", 30);
        v1.rent(5);
        v2.rent(4);
    }
}
class Vehicle {
    String model;
    double dailyRate;
    Vehicle(String model, double dailyRate) {
        this.model = model;
        this.dailyRate = dailyRate;
    }
    void rent(int days) {
        System.out.println(model + " rented for " + days + " days. Bill = " + (dailyRa
    }
}
class Car extends Vehicle {
```

OUTPUT

```
Sedan rented for 5 days. Bill = 400.0
```

QUESTIONS 20 / 20 completed

Resturant

RESTURANT

Write a Java program to implement a basic restaurant management system using inheritance. Create a parent class called Menu with attributes such as name and price. Then, create child classes such as Appetizer and Entree that inherit from the Menu class

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        Customer c = new Customer("Kaniga");
        Order o1 = new Order("Biryani", 250);
        Order o2 = new Order("Burger", 180);
        c.placeOrder(o1);
        c.placeOrder(o2);
        c.showTotal();
    }
}
class Restaurant {
    String name = "SIMATS Restaurant";

    void serve() {
        System.out.println("Serving...");
    }
}
class Order extends Restaurant {
    String item;
    int price;
    Order(String item, int price) {
```

OUTPUT

QUESTIONS 20 / 20 completed

Games

GAMES

Write a Java program to implement a basic game system using inheritance. Create a parent class called Game with attributes such as title and genre. Then, create child classes such as ActionGame and PuzzleGame that inherit from the Game class and have their

PROGRAM EDITOR

Java

```java
public class Main {
    public static void main(String[] args) {
        Player p1 = new Player("Hero", 100);
        Enemy e1 = new Enemy("Monster", 50);
        p1.attack(e1);
        e1.attack(p1);
        p1.displayStatus();
        e1.displayStatus();
    }
}
class Entity {
    String name;
    int health;

    Entity(String name, int health) {
        this.name = name;
        this.health = health;
    }
    void displayStatus() {
        System.out.println(name + " Health: " + health);
    }
```

OUTPUT

```
Hero attacks Monster!
```

QUESTIONS 20 / 20 completed

University

UNIVERSITY

Write a Java program to implement a basic university management system using inheritance. Create a parent class called Person with attributes such as name and age. Then, create child classes such as Student and Professor that inherit from the Person class

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        Student s = new Student("Kaniga", 1);
        s.display();
        Teacher t = new Teacher("Mr. John", "CSE");
        t.display();
    }
}
class UniversityMember {
    String name;
    UniversityMember(String name) { this.name = name; }
    void display() { System.out.println("Name: " + name); }
}
class Student extends UniversityMember {
    int id;
    Student(String name, int id) {
        super(name);
        this.id = id;
    }
    void display() {
        super.display();
```

QUESTIONS 20 / 20 completed

Library

LIBRARY

Write a Java program to implement a basic library management system using inheritance. Create a parent class called Item with attributes such as title and author. Then, create child classes such as Book and DVD that inherit from the Item class and have their

PROGRAM EDITOR

Java

```java
public class Main {
    public static void main(String[] args) {
        Book b = new Book("Java", "R1");
        b.display();

        Magazine m = new Magazine("Tech Today", 10);
        m.display();
    }
}
class LibraryItem {
    String title;
    LibraryItem(String title) { this.title = title; }
    void display() { System.out.println("Title: " + title); }
}
class Book extends LibraryItem {
    String author;
    Book(String title, String author) {
        super(title);
        this.author = author;
    }
    @Override
```

OUTPUT

```
Title: Java
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

QUESTIONS 20 / 20 completed

Shapes-1

SHAPES-1

Write a Java program to implement a basic shape hierarchy using inheritance. Create a parent class called Shape with attributes such as color and area. Then, create child classes such as Circle, Square, and Triangle that inherit from the Shape class and have their own unique

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        Shape redSq = new Square("Red", 5);
        Shape blueTri = new Triangle("Blue", 4);
        Shape greenCir = new Circle("Green", 3);
        redSq.draw();
        System.out.println(redSq.area());
        blueTri.draw();
        System.out.println(blueTri.area());
        greenCir.draw();
        System.out.println(greenCir.area());
    }
}
abstract class Shape {
    String color;
    Shape(String color) { this.color = color; }
    abstract double area();
    void draw() { System.out.println(color + " " + getClass().getSimpleName()); }
}
class Square extends Shape {
    double side;
```

OUTPUT

```
Red Square
```

INPUT

Your
here

QUESTIONS 20 / 20 completed

Animal-1

ANIMAL-1

Write a Java program to implement a basic animal classification system using inheritance. Create a parent class called Animal with attributes such as name and habitat. Then, create child classes such as Mammal, Reptile, and Bird that inherit from the Animal class and

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog("Rex");
        a1.eat();

        Animal a2 = new Cat("Kitty");
        a2.eat();

        Animal a3 = new Cow("Moo");
        a3.eat();
    }
}
abstract class Animal {
    String name;
    Animal(String name) { this.name = name; }
    abstract void eat();
}
class Dog extends Animal {
    Dog(String name) { super(name); }
    void eat() { System.out.println(name + " is a Carnivore"); }
}
```

OUTPUT

```
Rex is a Carnivore
```

QUESTIONS 20 / 20 completed

Employee

EMPLOYEE

Write a Java program to implement a basic employee management system using inheritance. Create a parent class called Employee with attributes such as name, id, and salary. Then, create child classes such as Manager and SalesPerson that

PROGRAM EDITOR

Java

Run Save

```java
import java.util.*;
class Employee {
    protected String name;
    protected int id;
    protected double salary;
    public Employee(String name, int id, double salary) {
        this.name = name;
        this.id = id;
        this.salary = salary;
    }
    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Salary: " + salary);
    }
    public void raiseSalary(double amount) {
        if (amount > 0) {
            salary += amount;
        }
    }
    public double getSalary() {
```

OUTPUT

QUESTIONS 20 / 20 completed

Bank

BANK

Write a Java program to implement a basic banking system using inheritance. Create a parent class called Account with attributes such as account number, account holder name, and balance. Then, create child classes such as SavingsAccount and

PROGRAM EDITOR

Java

Run Save

```java
public class Main {
    public static void main(String[] args) {
        SavingsAccount s = new SavingsAccount("Kaniga", 5000, 4.5);
        s.deposit(1000);
        s.withdraw(2000);
        s.applyInterest();
        s.display();
    }
}
class Account {
    String name;
    double balance;
    Account(String name, double balance) {
        this.name = name;
        this.balance = balance;
    }
    void deposit(double amt) {
        balance += amt;
    }
    void withdraw(double amt) {
        if (amt <= balance) balance -= amt;
```

OUTPUT

```
Name: Kaniga, Balance: 4180.0
```

INPUT

Your INPUT goes here!